

视频课程笔记

IBM 《Agent 运行时安全解析：如何保护非人类身份》

IBM 视频讲解 / AI 双语课 / Codex 整理

2026 年 5 月 28 日



视频作者 / 频道：AI 双语课

发布日期：2026 年 4 月 10 日

视频时长：12 分 25 秒

视频链接：<https://www.bilibili.com/video/BV1KADqBCEZJ>

目录

1 身份问题的人口：Agentic AI 也是非人类身份	1
2 从用户到后端资源：传统 IAM 的保护断点	4
3 四个运行时漏洞：问责、权限、委托与最后一跳	5
4 五项上线准入：把身份链路做成可审计闭环	8
5 三类支撑技术：编排、治理与跨团队可见性	11
6 可观测性的两面：态势管理与威胁管理	14

1 身份问题的人口：Agentic AI 也是非人类身份



图 1: 开场画面直接把主题写成 Agentic Runtime Security，说明本讲的重点落在智能体运行时的身份、访问与执行控制。¹

开场的关键词是 **Agentic AI**，也就是本笔记中统一称呼的**智能体式 AI**。讲者没有先谈模型能力、提示词技巧或应用场景，随即把问题放在 **agentic runtime security** 上：当智能体式 AI 被真正部署出来，它会在运行时代表某个调用方行动、访问后端系统、持有凭据、触发操作。安全讨论因此首先落到**身份与访问管理**（Identity and Access Management, IAM）上。

这一点很关键。许多组织谈到“身份”时，第一反应仍然是员工、管理员、外包人员、客户这类**人类身份**。讲者指出，智能体式 AI 进入企业系统后，天然属于**非人类身份**的一种。它不像员工那样坐在电脑前输入密码，但它会以代码、服务、容器、函数或虚拟机中的运行单元身份去执行动作。只要一个运行单元被赋予凭据、权限和可审计的主体标识，它就在 IAM 的视野内。

¹ 视频画面时间区间：00:00:09–00:00:16。

精确定义：身份与访问管理到底在管什么

在本章语境中，**身份与访问管理**可以理解为围绕一个行动主体建立的四件事：它是谁、如何证明自己、能访问什么、做过什么。用一个简化表达式表示：

$$\text{IdentityContext} = \langle \text{subject}, \text{credential}, \text{permission}, \text{audit_trace} \rangle.$$

其中 **subject** 是主体，可以是人，也可以是工作负载；**credential** 是密钥、令牌、证书等证明材料；**permission** 是被授予的访问能力；**audit_trace** 是后续追责与审计所需的行为痕迹。智能体式 AI 的特殊性在于：它经常以非人类主体运行，却又会代替人完成带有业务意图的动作。

1.1 热点话题背后的真正边界

视频一开始把智能体式 AI 称为当下 IT 中“最炙手可热”的主题，随后立刻收束到一个更窄的边界：如何部署带有运行时安全的智能体式 AI。这里的“运行时”指系统实际执行中的阶段。智能体不只停留在设计图、模型接口或演示页面里；它会在生产环境中接收请求、调用工具、读写数据、连接数据库或触发其他系统。

因此，本章的入口落在“AI 以什么身份行动”这个问题上。AI 是否聪明当然重要，但本段视频先处理部署后的身份边界。这类身份问题在传统 IAM 中并不陌生：机器账号、服务账号、微服务身份、容器身份都属于非人类身份。新的压力来自智能体式 AI 的行动范围更接近业务流程本身。一个 AI 代理可能读取上下文、调用多个服务、组合结果，并把后续动作交给另一个组件。它表面上像应用功能，安全上却是一个会移动、会调用、会扩展权限影响面的运行主体。

讲者提到 CISO 和 IT 团队时，指出他们谈身份往往先想到人类身份。这个反应本身可以理解，问题在于覆盖面不足。企业身份世界早已包含大量非人类主体；智能体式 AI 是其中更显眼、更容易放大风险的一类。如果 IAM 只在用户登录应用时生效，后续由代理继续发起的访问就会变成身份链条中的薄弱位置。

1.2 人类身份与非人类身份的比例失衡

讲者随后给出两组数字。第一组是“据称大约 80% 的网络攻击围绕身份或被攻破的身份展开”。第二组是“每一个人类身份，可能对应 45 到 90 个非人类身份”。这两组数字在本笔记中应作为**讲者引用的风险背景**来理解，不能当作所有行业、所有组织都必然满足的数学定律。

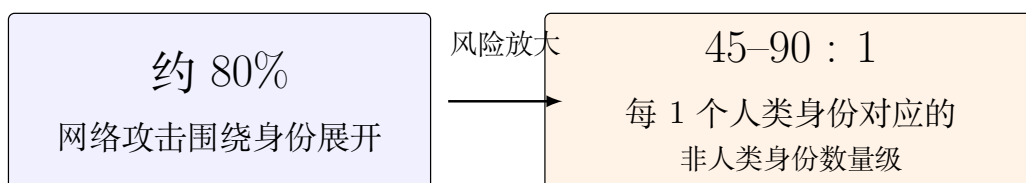
80% 这个说法的作用，是提醒听众：身份已经是攻击者常用的入口。攻击者拿到凭据、劫持会话、滥用权限、冒用账号时，系统往往会把这些动作看成“某个合法身份正在访问”。如果这个问题在人类身份侧已经足够严重，那么把视角扩展到非人类身份后，暴露面会快速扩大。

45 到 90 这个比例说明另一件事：现代系统里，真正发起访问的主体数量，远远超过组织结构图上的员工数量。每个员工背后可能有多个应用、微服务、流水线任务、容器、脚本、自动化作业和 API 客户端。智能体式 AI 被部署后，也会加入这张非人类身份网络。比例的具体值会因企业规模、云原生程度、自动化水平和治理成熟度不同而变化；讲者借它强调的是数量级压力。

如何读懂 80% 与 45–90

这两个数字的价值在于建立风险直觉，不能替代组织自己的资产盘点。80% 指向“身份是攻击路径高频入口”；45–90 指向“非人类身份规模可能远大于人类身份”。二者合在一起，形成一个务实判断：如果智能体式 AI 被当作普通功能上线，却没有进入非人类身份治理，组织会在

²根据 00:00:41–00:01:07 的讲述重绘。



讲者用这两个数字建立背景：身份已经是攻击高发面，智能体又把非人类身份的规模推到更高密度。

图 2：身份风险的两个尺度：攻击面集中在身份，非人类身份的数量又远高于人类身份。²

最拥挤的身份区域继续增加新主体。

1.3 工作负载视角下的 AI 代理

讲者对非人类身份给出的例子很具体：传统上，它可能是一个**工作负载**、一个微服务、一个容器。谈到 AI 时，讲者强调这些代理本质上属于这类工作负载。它们可以是 TypeScript 代码，也可以是 Python 代码；可以运行在容器中，也可以运行在 Lambda 函数或虚拟机上；组织随后会给它们提供身份，这就是“非人类身份”部分。

这段话把智能体式 AI 从抽象概念拉回工程现实。所谓 AI 代理，会落在具体系统里的可运行组件上。它要运行在某个计算环境里，要调用某个 API，要拿到某种令牌或证书，要穿过某个网络边界。对安全团队来说，关键问题因此变成：这个工作负载是谁创建的、在哪运行、拥有什么凭据、能访问哪些资源、访问时是否能被识别和追踪。

可以用一个类比理解：人类身份像办公楼里的员工证，非人类身份像机房里自动门禁系统、清洁机器人、传送带控制器各自持有的电子通行证。它们没有人的脸，也没有员工入职流程中的面谈环节，但它们会开门、搬运、读取或触发设备。智能体式 AI 的风险在于，它不只拿着通行证，还可能根据上下文决定下一步去哪里、调用谁、请求什么数据。

从工作负载视角看，Agentic AI 的安全问题不应只停留在“模型输出是否可靠”。运行时身份同样重要：一个代理若能访问 HR 系统、财务数据、客户库或主机资源，它的权限边界就必须像其他高风险工作负载一样被定义、收窄、记录和复核。讲者后面要讨论的“巨大漏洞”，正是组织把这些代理接进端到端 IAM 流程时容易留下的断点。

1.4 本章小结

本章完成了全篇的起点定义：Agentic AI，即智能体式 AI，在企业运行时环境中应被看作**非人类身份**的一种。它的工程形态是工作负载，可能由 TypeScript 或 Python 编写，运行在容器、Lambda 函数或虚拟机上，并被授予访问系统所需的身份材料。

讲者引用的 80% 身份攻击说法和 45-90 非人类身份比例，应该被理解为风险语境：身份攻击本来就是高频路径，非人类身份又在数量上快速扩张，智能体式 AI 会把这两个问题叠加到运行时链路中。下一章顺着这个判断继续推进：这些非人类身份到底在什么调用链上行动，以及传统 IAM 的保护边界会在哪里断开。

2 从用户到后端资源：传统 IAM 的保护断点

上一章已经把 agentic AI 放进非人类身份的范围里：AI 代理超出了单纯模型输出界面的范畴，它会作为运行时组件拿到某种身份，再替人或替应用去执行动作。本章只补上一块地图：这些身份到底沿着哪条链路移动，传统以人为中心的身份与访问管理（IAM）在链路上保护到哪里，风险又从哪里开始放大。讲者特别补了一句：这会影响“几乎所有”正在部署 agentic AI 的组织，说明它属于普遍部署风险，范围超过少数前沿团队。

2.1 调用链的基本形态

讲者先画出一条很直观的调用链，没有从复杂架构图切入。一个人类用户先打开或调用某个应用，应用再调用某个 AI 代理；在真实系统里，这个代理往往还会嵌入、转调或编排更多代理。链路继续向后走时，某一个代理最终会访问敏感资源，例如数据库、主机系统、企业内部 API，或其他承载关键数据的后端系统。

这条链可以写成：

$$H \rightarrow App \rightarrow A_1 \rightarrow \cdots \rightarrow A_i \rightarrow R$$

其中， H 表示 human identity，也就是发起请求的人类身份； App 表示被用户调用的应用； A_1 表示应用直接调用的第一个 AI 代理； A_i 表示链路中第 i 个代理实例， i 只是位置编号； R 表示后端资源，包括数据库、主机、外部系统或敏感数据源。这个式子不用于建立形式化安全模型，它只是把运行时路径压缩成一行，方便定位保护边界。

用口语说，这条链就是：人点了应用，应用叫来第一个代理，第一个代理可能再叫来别的代理，最后某个代理带着某种身份去碰真正有价值的数据。安全问题通常不在“是否有用户登录”这一点上结束，因为登录只说明链路的开头有人，无法自动说明后续每个代理的权限、意图和责任边界。



图 3: 讲者画出的运行时链路：用户进入应用，应用调用一个或多个 AI 代理，后续代理再触达敏感数据或后端系统。³

³ 视频画面时间区间：00:01:45–00:02:15。

2.2 传统 IAM 覆盖到哪里

传统的人类中心 IAM 更熟悉前半段链路：用户是谁，用户能不能进应用，应用能不能代表这个用户启动第一个代理。这一层保护通常围绕人类身份展开，例如认证、单点登录、会话、用户组、角色和应用入口权限。按照讲者的说法，它可以保护到第一个 AI 代理这一点。

精确地说，保护边界在 $H \rightarrow App \rightarrow A_1$ 附近。这里的“保护”含义有限：系统能够确认某个用户通过某个应用触发了第一个代理，或者能够把入口访问控制施加在应用和第一跳代理上。它还没有自动回答后面的问题：第一个代理转调第二个代理时，第二个代理是否继承了正确身份；第 i 个代理访问 R 时，权限是否仍然最小；后端资源看到的调用者是否还能被追溯到具体人、具体代理和具体动作。

这就是本章标题里的“断点”。断点的含义很具体：IAM 没有完全失效，但它的默认视角停在了人类用户和入口应用附近。运行时链路越长，后续代理越多，入口处的身份判断越难代表最后一跳的真实风险。把门禁设在大楼入口，能够知道谁进了大楼；如果楼内还有机器人、代办人员、自动门和资料室钥匙，入口门禁无法天然解释每一次资料室访问。

2.3 嵌入式代理带来的后端风险

风险开始明显上升的位置，是嵌入式代理继续向后端资源发起调用的阶段。这里的“嵌入式代理”指应用内部或代理链路内部被调用的 AI 代理组件。它们可能不直接面对用户，却能读写数据、调用工具、连接系统，甚至触发跨系统动作。对安全团队来说，这类组件已经具有非人类身份的特征：它没有自然人属性，却会持有凭证、权限和运行时上下文。

关键风险来自身份语义的丢失。用户 H 的身份、应用 App 的业务上下文、第一个代理 A_1 的任务目标，传到 A_i 时可能被压扁成一个过宽的服务账号、一个长期有效的令牌，或一个看起来像用户本人的调用。后端资源 R 只看到“有权限的请求”，却未必看得到这次请求是哪个代理发起、受谁委托、为了哪个动作、是否仍处在原始用户意图范围内。

数据库和主机系统在这里很典型。它们通常承载高价值数据，访问控制也更依赖精确身份。如果嵌入式代理用宽权限连接数据库，链路前端的登录状态就变成了弱证据；如果代理沿用用户身份去访问主机，审计记录里可能只留下用户名字，代理的中间动作被擦掉；如果代理之间互相调用，安全策略还需要区分“这个代理能调用工具”和“这个代理能读取敏感表”这两类差异。

所以，本节的保护边界应当画得很窄：人类中心 IAM 覆盖到第一个代理，后续嵌入式代理访问敏感资源时，需要新的运行时身份、权限、委托和审计机制。第 3 章要展开的四个漏洞，正是从这个边界之后长出来的：谁负责、权限多大、谁委托了什么动作，以及最后访问资源时有没有重新检查。

2.4 本章小结

本章把运行时路径压缩成一条链： $H \rightarrow App \rightarrow A_1 \rightarrow \dots \rightarrow A_i \rightarrow R$ 。它提醒读者，agentic AI 的身份风险不只发生在用户登录处，也发生在代理继续代表应用或用户访问后端资源时。

传统人类中心 IAM 的有效覆盖范围应当谨慎描述为到第一个 AI 代理为止。风险集中出现在更深的嵌入式代理链路：当 A_i 连接数据库、主机或其他敏感资源 R 时，系统必须重新说明身份、权限、委托关系和资源侧执行检查。下一章会沿着这个断点，拆解运行时里的四个具体漏洞。

3 四个运行时漏洞：问责、权限、委托与最后一跳

上一章把路径画清楚了：人类用户进入应用，应用再调用一个或多个 AI 代理，代理继续访问数据库、主机或其他后端资源。讲者说的“四个 crucial holes”，就发生在这条链路穿过第一个代理之后。

传统身份与访问管理可以把人和应用之间的入口守住，但代理作为非人类身份继续行动时，还需要回答更细的问题：谁在做、拿了多少权限、凭什么代表人行动、最后触碰敏感资源时有没有再次检查。

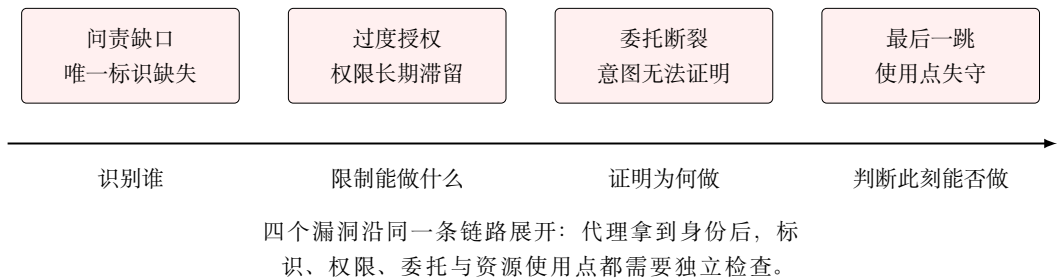


图 4: 四个运行时漏洞的关系图：它们共同决定代理行为能否被识别、限制、解释和拦截。⁴

3.1 问责缺口：每个代理实例都要能被叫出名字

问责能力，在本节中指把一个运行中的 AI 代理、它的具体实例以及它执行过的动作关联起来的能力。这里避开抽象责任口号，重点放在可追踪的运行时事实上：哪个代理实例在什么时间调用了什么资源，执行了什么动作，留下了什么日志线索。若用简化记号表示，链路中的代理可以写作 A_i 。问责能力要求系统能把动作绑定到具体的 A_i ，不能只看到一团模糊的“应用行为”。

唯一标识符就是这个绑定的起点。它是分配给代理或代理实例的 ID，使系统能够区分“这个 HR 代理实例”和“另一个 HR 代理实例”。讲者强调，给 AI 代理提供 accountability 时，需要给它们某种 identifier ID，让它们保持唯一，这样才知道每个实例是什么，并能追溯它正在做什么。直观地说，唯一标识符像工牌和门禁记录的组合：工牌让人知道进入者是谁，门禁记录让人知道它走过哪里。

如果没有唯一标识符，后续的审计日志也会失去抓手。日志里可能只剩下“某个服务访问了数据库”，安全团队无法判断这是哪个代理实例、由哪次会话触发、是否符合当时的任务。问责缺口因此是其他漏洞的底座：一旦 A_i 这个主体不可见，权限、委托和最后一跳检查都会变得含混。

3.2 过度授权：静态宽权限会把代理变成常开钥匙

过度授权指 AI 代理持有超过当前请求、当前动作或当前会话所需范围的权限。讲者给出的例子很具体：一个 HR 代理可能被授权入职或离职员工，但这项权限不应在代理运行的全部时间里都存在。合理边界应当收缩到某次请求、某个动作和某个会话。

最小权限是控制这个问题的原则：主体只获得完成当下任务所需的最小访问范围。若代理在会话 s 中执行动作 $action$ ，它的权限可以粗略写作 $P(A_i, s)$ 。最小权限要求 $P(A_i, s)$ 随任务收窄，避免长期附着在代理身上。这里的关键不在于权限名称多么精细，关键在于权限生命周期：该用时出现，动作结束后撤回。

零信任在本段语境中是另一条配套原则：系统不因代理处在应用内部、由可信用户触发、或已经启动运行，就默认它可以继续访问敏感资源。每次请求和每个动作都需要结合上下文与风险重新判断。它像机场安检后的登机口复核：通过第一道门不代表后面的每道门都自动放行。

静态宽权限的风险

如果开发者不清楚代理真正需要哪些权限，代理很容易逐步“拿宽”：先为了跑通流程授予一组权限，再因为没人审计、没人回收而长期保留。讲者指出，这类代理会在无人查看、无人审计、

⁴根据 00:02:29–00:05:40 的讲述重绘。

无人释放权限的情况下变成过度授权状态。问题的锋利处在于，代理运行速度快，权限又常驻，错误访问可能早已发生。

3.3 委托与意图：Tyler 到 HR 代理的授权链

委托指人类用户授权 AI 代理代表自己执行某个具体动作。委托的核心，是把“谁允许哪个代理做哪件事”说清楚，避免把人的全部身份交给代理无限期使用。讲者用 Tyler 的例子说明：如果 Tyler 在 HR 部门，他可以委托一个 HR 代理去操作某件 HR 事务。这里的 Tyler 同时是对谈中的讲者之一，视频借他的名字代入 HR 专业人员场景，用来说明人类身份、代理身份和动作意图如何绑定。安全问题随即出现：系统怎样确认这是 Tyler 对这个 HR 代理的有效委托，并且这个委托对应的是某个明确动作。

意图指委托背后的具体目的和动作边界。它回答的问题是：Tyler 想让 HR 代理做什么，不能停留在“代表 Tyler”这种泛泛说法。本章先把意图理解为可被记录的业务目的：某个 HR workflow、某项员工操作、某次会话中的授权边界。更精确的三元绑定会在下一章作为上线准入公式展开。

这一点解释了讲者为什么把 delegation、intent 和 audit logging 放在一起讲。委托需要让代理获得行动资格，意图让行动资格变窄，审计日志让事后可以复盘。三者合在一起，才有可能回答“Tyler 是否授权这个 HR 代理在这次会话里执行这项 HR 操作”。

3.4 身份冒用：代理继承用户身份会抹掉代理本身

身份冒用指 AI 代理不使用自己的非人类身份与唯一标识符，转而继承触发它的人类用户身份来访问系统。讲者把这种代理称作有点“懒”：它不为自己获取唯一标识符，直接沿用调用者的身份。访问检查可能因此通过，因为系统看到的是用户身份；但问责能力会立即受损，因为日志里很难分辨到底是用户本人操作，还是代理代为操作。

协作型代理让这个问题更直观。它们可能运行在桌面或浏览器环境中，以用户的姿态点击、浏览和提交。安全视角下，最麻烦的问题就是：这是代理在操作，还是用户在操作？如果系统没有代理自己的身份，后端只会看到用户，代理这个行动主体被擦掉了。

身份冒用会吞掉问责链

一旦代理冒用用户身份，访问成功本身不能证明行为合理。系统可能只记录 Tyler 访问了某个资源，却看不到 HR 代理是否介入、是否被明确委托、是否执行了原定动作。这种记录看似完整，实际缺少最关键的非人类身份维度。

3.5 最后一跳：触碰敏感资源时还要再问一次

最后一跳指 AI 代理从自身运行环境走向敏感后端资源的那一次访问，例如从代理到数据库、主机或外部系统的连接。讲者用 last mile 来描述这个问题；本章统一称为“最后一跳”，强调资源使用点附近的检查，登录入口处的校验无法替代这一层判断。

最后一跳的风险来自几个条件叠加：代理可能已经过度授权，可能正在冒用用户身份，可能共享同一个数据库凭据，还可能以机器速度连续访问敏感数据。如果此时系统没有在资源调用发生的那一刻检查上下文和风险，就会错过最基本的控制点。讲者的直觉判断很直接：当代理调用敏感数据时，应当有人检查这次访问在当前上下文和风险因素下是否仍然准确、合适。

讲者担心的还有时间错位：代理真正访问敏感数据时，可能已经离最初授权或流程入口很远，安全系统若只记入口状态，就无法判断当前访问仍然合适。最后一跳还包含撤销能力：若代理共享数据

库凭据，且风险模型判断当前访问异常，系统必须能在代理以机器速度操作时分析并撤销其最后一跳访问。

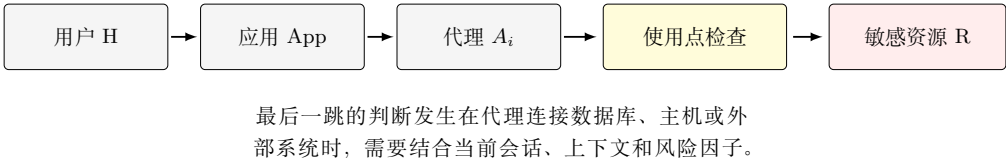


图 5: “最后一跳”把授权判断推到资源使用点，防止链路前端的通过被误当作后端访问许可。⁵

图示强调的是判断位置：检查应贴近数据库、主机或外部系统的使用点，而不能只停在应用入口。第 4 章会把这件事写成准入公式，展示代理身份、会话权限、人的委托意图和当前风险怎样进入同一个允许/拒绝判断。

3.6 本章小结

本章把运行时漏洞收束为四类可复用问题。问责能力要求每个代理实例有唯一标识符，并能追踪其动作；过度授权描述代理拿到超过当前请求和会话所需的权限，最小权限与零信任要求权限随请求、动作和风险动态收缩；委托与意图要求把人、代理和动作绑定在一起，身份冒用则是这条委托和问责链断裂后的典型表现；最后一跳要求在敏感资源使用点附近做近实时检查。

下一章的五项上线准入要求，正是对这些漏洞的逐项回应：注册代理解决问责，剥离常驻权限解决过度授权，绑定意图解决委托漂移，在使用点强制执行解决最后一跳，完整审计则把人类身份、非人类身份、动作和响应重新接成可核查的链路。

4 五项上线准入：把身份链路做成可审计闭环

上一章把运行时缺口拆成了问责、过度授权、委托意图与最后一跳四类问题，其中身份冒用是委托和问责链断裂后的典型表现。到了上线准入阶段，问题要从“哪里会断”推进到“交付前必须证明什么已经接上”。讲者把这组要求称为 AI 代理进入市场的五项 imperative，也可以理解为一个 AI 项目的 definition of done：没有这些检查，代理可以运行，但安全责任链仍然是散的。

这里的“上线准入”不只是一张发布清单。它更像机场登机口的核验：人、票、行李、目的地、当前风险都要在同一个场景里对上。AI 代理的核验对象则是人类身份、非人类身份、会话权限、动作意图、资源使用点和审计证据。任何一项脱节，第三章的漏洞都会在生产环境里重新出现。

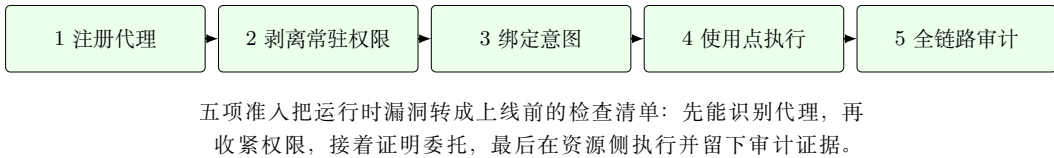


图 6: AI 代理上线的五项准入要求，构成从身份登记到审计闭环的交付标准。⁶

4.1 注册代理：先让每个执行者进入名册

第一项准入是注册代理。所有进入运行时链路的 AI 代理都要有登记记录、可识别身份和基本风险画像。这里的登记并非只写一个服务名，还要回答三个问题：这个代理是什么，它以什么身份行动，

⁵根据 00:04:35–00:05:40 的讲述重绘。

⁶根据 00:05:40–00:08:47 的讲述重绘。

它会连接哪些外部系统。

这直接补上第三章的问责缺口。没有注册，代理实例就像没有车牌的车辆；出事以后只能知道“某个系统访问了资源”，很难追到哪个代理、哪个实例、哪个会话发起了动作。注册提供的是问责链的起点：唯一标识、所属应用、运行环境、外部连接范围和风险等级。讲者的判断更直接：登记阶段应把外部连接视为默认风险，因为这些代理基本都会发生外部连接。尤其当代理会访问数据库、主机、云资源或第三方接口时，风险量化要在准入阶段完成，因为这些外部连接正是后续访问控制与审计的边界。

注册也要避免一个常见误判：把代理看成应用内部的小功能。代理一旦能代表工作流行动，它就具有非人类身份属性。它可以被授权、可以调用资源、可以产生业务后果，所以它需要独立进入身份名册。否则，后续的权限剥离、意图绑定和使用点强制执行都缺少对象。

4.2 剥离常驻权限：默认不保留可长期调用的权力

第二项准入从权限开始：剥离常驻权限。常驻权限指一项能力在具体会话、具体动作结束后仍然长期可用，例如某个 HR 代理一直拥有入职或离职员工的能力。它的问题不在于某个权限本身一定错误，关键在于权限脱离了当前任务，变成可被随时调用的库存。

这对应第三章的过度授权缺口。传统人类身份管理常按角色授予长期权限，例如“某用户属于 HR 角色，所以未来一两年都拥有某组访问权”。对 AI 代理套用这种模式会放大风险，因为代理运行速度快、调用频率高、可被工作流自动触发，并且可能同时接触多个后端系统。一个人类用户的多余权限也危险；一个自动化代理的多余权限会把错误或攻击放大成批量动作。

剥离常驻权限的实际含义是：代理上线时不应携带一包长期有效的通用权限。可以把它想成仓库钥匙管理。人类员工也许长期持有办公室门禁，但临时进入金库仍要单独审批、登记、限时。AI 代理访问敏感资源时也应如此：常规身份存在，敏感能力不常驻。

4.3 会话级动态权限：把授权压缩到当下任务

剥离常驻权限之后，还要有可操作的替代机制：会话级动态权限。动态权限是在具体会话、具体任务、具体上下文中即时授予的权限，任务结束后收回。它把“代理能做什么”从静态角色表转成实时判断。

这仍然回应第三章的过度授权，也进一步压低身份冒用的伤害面。即便某个代理被误用或被诱导执行异常动作，它能够触达的资源也被限定在当前会话授予的范围内。讲者用 HR 场景说明这一点：代理不应一直拥有入职或离职员工的能力；只有当 Tyler 作为 HR 专业人员发起相应工作流，并且当前会话满足条件时，代理才获得相应能力。

可以用一个直觉公式表达这类判断，但它只承担教学简写的作用，不能当作正式安全模型：

$$Allow(A_i, R, action, t) = Policy(P(A_i, s), I(H, A_i, action), Risk(A_i, R, t))$$

其中， A_i 表示第 i 个代理实例； R 表示后端资源，例如数据库、主机或外部系统； $action$ 表示代理理想执行的具体动作； t 表示动作发生的时间； s 表示当前会话； $P(A_i, s)$ 表示代理 A_i 在会话 s 中被授予的权限； H 表示发起或授权该工作流的人类身份； $I(H, A_i, action)$ 表示人类身份、代理实例与动作之间的意图绑定； $Risk(A_i, R, t)$ 表示代理在时间 t 访问资源 R 的上下文风险； $Policy(\dots)$ 表示组织策略对这些输入的综合判定； $Allow(\dots)$ 表示最终是否允许该动作。

这个公式的价值在于提醒读者：授权不应只看“代理是谁”。它还要看当前会话授予了什么权限、谁让它做这件事、它访问什么资源、当前风险是否可接受。

4.4 将行为绑定到意图：证明谁让哪个代理做了哪件事

第三项准入是把所有动作绑定到意图。这里的意图指一组可记录、可验证的委托关系：哪个人类身份，在什么业务上下文中，要求哪个代理执行哪一个动作。

这补的是第三章的委托缺口，也会影响身份冒用问题。没有意图绑定时，系统可能只看到代理执行了某个动作，或者只看到 Tyler 登录过应用，却无法证明 Tyler 授权 HR 代理 X 在某个工作流里执行入职或离职操作。讲者把这称为审计问题的关键，因为代理已经不只处理 HR 流程，还可能处理银行交易、银行转账，以及云上和本地基础设施的创建与配置。

意图绑定要把三段证据接起来：人类身份、代理身份、业务动作。类比合同签署，只有“某人签过字”还不够，还要知道签的是哪份合同、由谁代办、代办权限覆盖哪些条款。AI 代理场景也是如此：只有当 Tyler、HR 代理、员工离职动作、工作流编号和会话上下文被连成一条证据链，审计人员才能判断这是被授权的代理行为，还是代理越权、被误导或冒用了用户身份。

4.5 在使用点强制执行：最后一跳也要重新判定

第四项准入是在使用点强制执行。使用点指代理真正接触数据库、主机、外部 API 或其他后端资源的那一刻。讲者把这称为最后一英里，也称最后 100 英尺、100 米，最后归结为最后一跳。无论叫法如何，核心都是同一件事：代理连接资源的瞬间要做接近实时的策略或风险分析。

这直接回应第三章的最后一跳缺口。只在代理部署时审批一次，不能证明它在一个月或两个月后的某次会话中仍然应该访问同一个资源。上线时被允许，不能自动推出当前动作也应被允许。使用点强制执行要求系统在资源连接处发问：这个代理此刻连接这个数据库或主机，执行这个动作，是否仍然被授权。

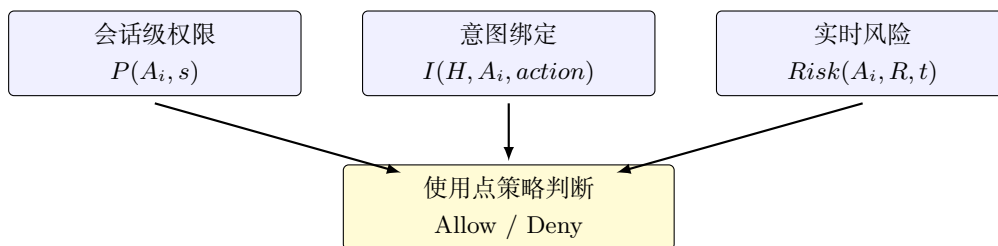


图 7：使用点强制执行需要把权限、意图和实时风险合并判断，才知道本次访问能否继续。⁷

这一控制还会反向约束前面几项准入。注册提供代理身份，动态权限提供会话边界，意图绑定提供委托证据，风险分析提供当前上下文。使用点像门锁本身，前面的登记、审批和路线规划都重要，但真正决定能否进入敏感房间的检查发生在门口。

4.6 全链路可审计：从人到代理再到响应都要留下证据

第五项准入是控制证明，也就是全链路可审计。对高度监管行业来说，这一点尤其关键。讲者明确提到医疗健康、生命科学和金融服务：这些行业把产品或代理能力推向市场时，不能只声称有控制措施，还要能够拿出控制证据。

这把第三章的问责缺口、委托缺口、身份冒用和最后一跳问题收束到同一条审计链上。过去只审计“人类用户到应用”的路径已经不够。完整链路要覆盖人类身份、非人类身份、代理动作和系统响应。换成链式表达，就是：

⁷ 根据 00:07:25–00:08:05 的讲述重绘。

$$H \rightarrow App \rightarrow A_1 \rightarrow \cdots \rightarrow A_i \rightarrow R \rightarrow response$$

其中, H 是人类身份, App 是人类调用的应用, A_1 到 A_i 是一个或多个代理实例, R 是后端资源, $response$ 是资源访问后的结果或反馈。审计要能沿着这条链回答: 谁发起, 哪个应用承接, 哪些代理参与, 每个代理拿到什么会话权限, 基于什么意图执行了什么动作, 在哪个使用点被允许或拒绝, 最后产生了什么响应。

全链路可审计的难点在于连续性。日志碎片很多组织都有, 但碎片不能自动构成证据。只有当身份标识、会话、权限、意图、资源访问和响应可以被同一个追踪关系串联起来, 审计才有证明力。否则, 事故复盘会停在“某个应用调用了某个系统”这种粗粒度描述上, 无法支撑监管、客户问责或内部整改。

4.7 本章小结

本章把第三章的运行时漏洞转成五项上线准入: 注册代理, 剥离常驻权限并采用会话级动态权限, 将行为绑定到意图, 在使用点强制执行, 以及建立全链路可审计性。它们共同形成一个闭环: 先知道代理是谁, 再限制它在当前会话能做什么, 随后证明谁让它做这件事, 并在最后一跳重新检查, 最终留下从人类身份到非人类身份、动作和响应的连续证据。

下一章需要回答实现问题: 要让这些准入要求在真实企业环境里持续运转, 单靠写策略文档不够, 还需要编排、治理和可观测性等技术支撑。

5 三类支撑技术: 编排、治理与跨团队可见性

上一章把上线准入拆成了五项可执行要求: 注册代理; 剥离常驻权限并采用会话级动态权限; 把行为绑定到意图; 在使用点强制执行; 留下完整审计证据。到这里, 问题从“应该检查什么”进入“企业怎样长期跑起来”。讲者给出的答案很直接: 要让这些控制跨越人类身份与非人类身份, 需要三类技术共同工作, 分别是编排、治理和可观测性。

这三者处理的层次不同。编排负责把调用路径串起来, 像交通调度员一样让身份、会话、代理和资源访问按正确顺序流动; 治理负责把策略贴到每一步, 确保从入口访问到最后一跳都有规则可查; 可观测性负责让 CISO、IT 和开发团队看见同一张运行图, 避免安全目标、运维目标与交付目标各说各话。

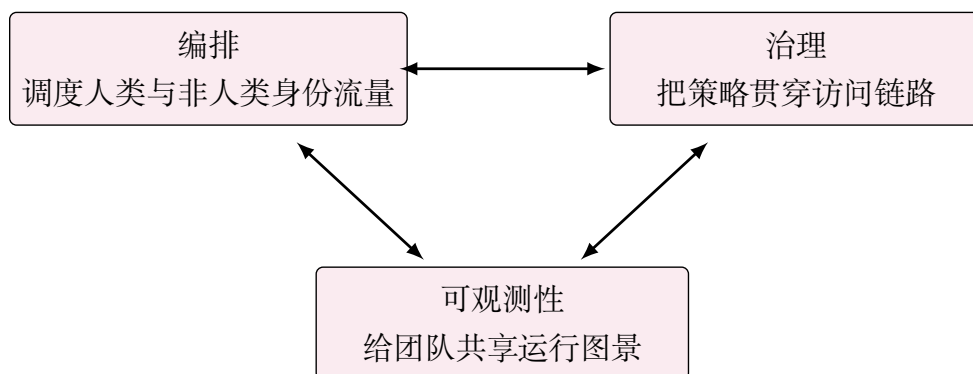


图 8: 讲者提出的三类支撑技术: 编排、治理和可观测性需要协同覆盖人类与非人类身份。⁸

⁸根据 00:08:47–00:09:40 的讲述重绘。

5.1 编排引擎：在两类身份世界之间调度流量

讲者首先提到的是编排。这里的编排不能只理解成工作流自动化。工作流自动化关心任务先后顺序，例如先调用 HR 系统，再发送通知；本讲中的编排还要处理身份链路：人类身份进入应用，应用触发一个或多个 AI 代理，代理再访问数据库、主机或外部系统。每一跳都涉及“谁在发起”“谁在代理”“拿到什么权限”“访问哪个资源”。

可以把编排引擎想成企业机场里的塔台。跑道上既有人类身份的航班，也有非人类身份的航班；它们速度不同、载荷不同、风险不同，却会经过同一批后端资源。塔台的价值在于统一指挥：哪一个代理实例接到请求，哪一个会话授予临时权限，哪一个使用点需要重新检查，哪一段动作需要写入审计证据。

编排在本讲中的核心含义

编排负责管理人类身份世界与非人类身份世界之间的运行时流量。它的目标并非单纯把流程跑通，还要把可审计性和问责能力嵌入调用路径，使 $H \rightarrow App \rightarrow A_i \rightarrow R$ 这条链路上的主体、权限、意图和资源访问能够被串联起来。

这也解释了为什么上一章的五项准入要求需要技术底座。这五项应合并理解为：注册代理；剥离常驻权限并以会话级 just-in-time 动态权限替代；将所有动作绑定到意图；在使用点强制执行；提供控制证明与全链路审计。注册代理需要一个地方接收、标识和调用代理；动态权限需要一个机制按会话授予与回收能力；意图绑定需要把人、代理和动作同时放进同一条运行记录；最后一跳检查需要在资源访问发生时取得当前上下文。没有编排，这些控制容易变成分散脚本或人工审批，难以支撑高速运行的代理系统。

5.2 治理框架：把策略压到每一次访问和最后一跳

第二类技术是治理。讲者说得很具体：从这里的访问，一直到最后一跳、最后缺口，无论怎样命名，每一步都要应用治理，这样组织才能证明“谁做了什么”。治理在这里的含义偏运行时策略管理，重点放在策略如何覆盖每一次访问，而非停留在制度文本。

治理要回答四类问题。第一，哪些主体可以进入链路，例如哪些用户、应用、代理实例已被登记。第二，哪些权限可以在当前会话中出现，例如代理是否能临时访问某个 HR 操作或数据库表。第三，哪些动作与人类意图绑定，例如 Tyler 是否授权某个 HR 代理执行某项具体 HR 动作。第四，资源使用点是否仍然允许访问，例如代理此刻访问主机或数据库的风险是否可接受。

用教学公式压缩，治理可以看成对上一章授权判断的策略层：

$$Decision = Governance(Policy, Identity, Privilege, Intent, Risk)$$

其中，*Policy* 表示组织策略；*Identity* 包含人类身份与非人类身份；*Privilege* 表示会话级权限；*Intent* 表示人、代理与动作之间的意图绑定；*Risk* 表示当前上下文风险；*Decision* 表示允许、拒绝、升级审批或记录告警等结果。

治理与审计证明的关系

治理的结果不能只存在于策略引擎内部。它需要留下可审计证据：哪个策略参与了判断，输入上下文是什么，最终为什么允许或拒绝。否则，组织在事后只能说“系统当时放行了”，很难证明放行符合当时的人、代理、动作和资源条件。

治理的难点在于跨度。人类身份管理通常熟悉登录、角色和应用入口；非人类身份管理还要处理工作负载、服务凭证、代理实例、密钥材料和后端资源调用。AI 代理把这两个世界叠在一起，导致策略范围从“用户能否进入应用”扩展到“代理在当前会话能否代表某个人执行某个动作，并在最后一跳访问某个资源”。治理框架必须覆盖这整段路径。

5.3 组织协作断层：CISO、IT 与开发的目标并不相同

讲者在第三类技术之前先谈组织问题，因为这正是很多企业卡住的地方。围绕 agentic runtime security，CISO 办公室、IT 团队和开发团队都参与其中，但三方关心的事不同。开发团队想把能力尽快推到业务里，推动产品和流程前进；IT 团队要让系统可管理、可运行、可维护；CISO 关注风险、控制和事故责任。

这种差异本身合理，问题出在三方缺少共同可见的事实层。讲者举了一个很具体的客户对话：他们问 CISO，“你们和开发团队在做什么？”对方回答，大意是每个月开一次会，讨论应该一起推进哪些事项。接着再问，“下个月你们做什么？”对方几乎给出同一句回答：还是每月开会，继续讨论应该一起推进哪些事项。讲者特别提到，对方说这话时还会带着一丝笑意，因为所有人都知道这种会议并没有真正形成持续协作机制。

月度会议轶事（约 00:10:07–00:10:27）

提问方：你们和开发团队在做什么？

CISO 侧回答：我们有一个月度电话会议，讨论应该一起做哪些倡议。

追问：那下个月做什么？

回答：我们有一个月度会议，讨论应该一起做哪些倡议。

这个轶事的刺痛点在于：会议存在，协作未必存在；议题存在，共同事实未必存在。CISO 可以提出风险要求，IT 可以提出运维边界，开发可以提出交付压力，但如果三方看不到同一批代理、同一批权限、同一批秘密材料、同一批最后一跳访问事件，讨论就会停留在倡议层面。

5.4 可观测性作为协作基础：让三方共享同一张运行图

讲者把第三类技术留给可观测性，并称其最有意思，原因就在这里。可观测性在本章中的角色，首先是跨团队可见性。它让 CISO、IT 和开发围绕同一张运行图协作：哪些 AI 代理正在运行，哪些代理有唯一标识，哪些代理访问了后端资源，哪些秘密管理器、密钥管理服务和证书管理器在被使用，哪些访问绕过了预期路径。

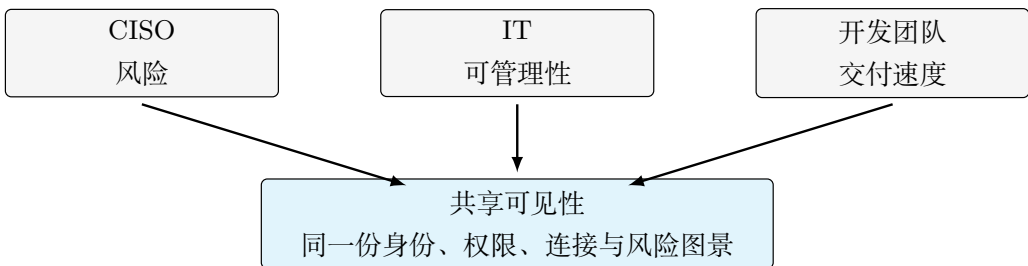


图 9：跨团队协作的关键在于共享可见性，否则每月会议只能同步项目名称，无法共同定位运行时风险。⁹

⁹根据 00:09:40–00:10:44 的讲述重绘。

可观测性不能等同于把日志堆在一起。日志是原材料，可观测性是把原材料组织成可以行动的视图。对 CISO 来说，它要呈现风险：代理是否绕过身份登记，是否持有过宽权限，是否缺少最后一跳检查。对 IT 来说，它要呈现可管理性：身份管理引擎、秘密管理实例、密钥与证书路径是否分裂。对开发来说，它要呈现交付影响：哪些控制是上线前必须满足的，哪些异常会阻断代理调用，哪些问题可以在开发阶段提前修复。

只有会议，没有共同可见性，会让治理停在口头层

agentic runtime security 横跨应用开发、运行平台和安全治理。若三方只能定期开会，却没有同一张代理身份与资源访问图，协作会反复回到“我们应该一起做什么”的讨论。真正推动落地的抓手，是让每个团队看到同一批事实，并能把事实映射到各自责任。

这也为下一章埋下了边界：本章把可观测性作为组织协作基础，下一章会进一步拆开可观测性的两面。第一面是态势管理，关注当前身份管理和秘密管理格局是否已经失控；第二面是威胁管理，关注运行时是否出现绕过秘密管理器、缺少唯一标识符等异常代理行为。

5.5 本章小结

本章从上线准入要求转向实现技术。编排解决调用链路的调度问题，把人类身份、非人类身份、会话权限、意图和资源访问放进同一条运行路径；治理解决策略覆盖问题，让每一步访问到最后一跳都可以被规则约束并留下证明；可观测性解决跨团队协作问题，让 CISO、IT 和开发围绕同一张事实图工作。

讲者的月度会议轶事说明，组织卡点往往并非完全缺少沟通，真正缺口在于缺少可共享、可运行的运行时事实。下一章将把可观测性拆成态势管理与威胁管理，说明它怎样同时服务于风险盘点和实时异常发现。

6 可观测性的两面：态势管理与威胁管理

上一章把可观测性放在跨团队协作的位置上：CISO、IT 和开发团队需要看见同一张运行图。讲者随后进一步拆解：围绕 agentic runtime security，可观测性有两面，一面是态势管理，另一面是威胁管理。前者回答“当前格局有多乱、风险积累在哪里”，后者回答“运行时是否已经出现绕过控制的异常行为”。

这一区分很关键。很多企业以为只要有日志，就等于看见了风险；实际情况更硬：日志可能散在不同工具里，身份管理器可能各自为政，秘密、密钥和证书可能由不同团队用不同实例维护。可观测性要把这些碎片拉到同一张图上，才能支持改进和检测。

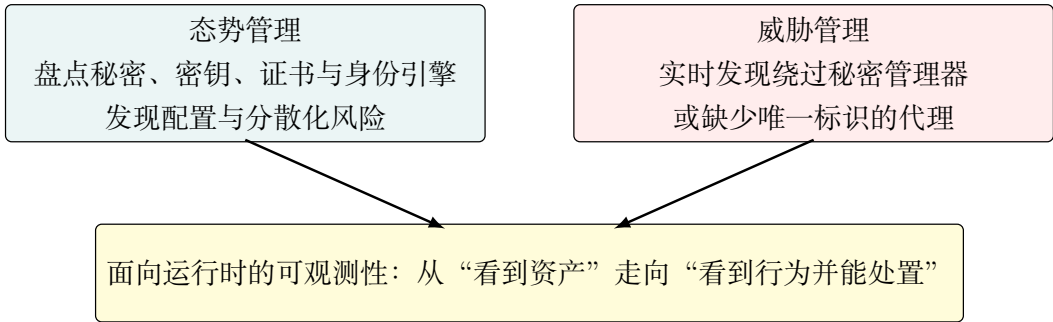


图 10: 可观测性的两面：态势管理回答“有什么和在哪里”，威胁管理回答“此刻谁在异常访问”。¹⁰

6.1 态势管理：先看清秘密、密钥和证书管理的分裂

态势管理关注的是当前配置、资产和管理面貌。讲者给的例子非常具体：如果开发团队手里有 13 个不同的 secrets manager 或实例，例如社区版 secrets manager，再加上云特定的 key manager 和 certificate manager，这会变成风险管理问题。这里必须把三个例子分清。

secrets manager 管理的是应用和代理运行时需要取用的秘密材料，例如 API token、数据库密码、服务凭证。它关心“秘密放在哪里、谁可以取、什么时候取、取用是否留下记录”。

key manager 管理的是加密密钥，例如云服务中用于加密数据、签名请求或保护存储的 key。它关心“密钥怎样生成、轮换、授权使用和吊销”。

certificate manager 管理的是证书，例如服务之间 TLS 连接使用的证书。它关心“证书由谁签发、何时过期、部署到哪里、哪些服务依赖它”。

为什么这三类管理器不能混成一个词

秘密、密钥和证书都属于机器身份与访问链路中的敏感材料，但生命周期和失败方式不同。秘密泄露会让代理直接调用系统；密钥失控会影响加密与签名边界；证书过期或误发会破坏服务间信任。讲者把三类例子并列列出，重点在于说明管理面已经碎片化，风险因此难以统一评估。

态势管理要做的第一步，是把这些实例盘出来：有多少个非人类身份管理引擎，多少个 secrets manager 实例，多少个云特定 key manager，多少个 certificate manager，它们分别被哪些团队、应用和代理使用。看不见这些分裂，组织就容易默认“反正各团队都有工具”，然后忽略整体风险。

6.2 非人类身份管理引擎盘点：从看见数量到推动收敛

讲者说，直到你看见这个问题之前，它很容易被忽略。态势管理的价值就在于把“工具很多”变成可讨论、可治理的事实。它不只列清单，还要回答这些管理引擎的边界：哪些代理从哪个管理器取秘密，哪些工作负载用哪个密钥服务，哪些证书支撑哪些服务调用，哪些路径缺少统一策略。

这类盘点会自然引出收敛问题。若一个组织同时运行十几个秘密管理实例，风险并不只来自数量多，还来自策略不一致、轮换周期不一致、审计格式不一致、权限模型不一致。开发团队可能为了速度引入社区版工具，云团队可能使用云原生 key manager，平台团队可能维护证书管理器；每个选择都有局部理由，组合起来却会增加全局复杂度。

态势管理的判断标准

态势管理要让组织回答两个问题：第一，当前有多少个非人类身份与敏感材料管理入口；第二，这些入口是否需要整合、替换或纳入统一治理。它的核心价值是把散落的管理面转成可改进的风险图。

在 agentic AI 场景里，这个问题会被放大。AI 代理调用速度快，可能跨多个后端系统取用秘密，且经常由开发团队快速试验和上线。如果秘密管理、密钥管理和证书管理已经分裂，代理会沿着最容易接入的路径前进，安全团队事后才发现某些代理绕过了预期身份链路。

6.3 威胁管理：实时发现绕过秘密管理器的代理

可观测性的第二面是威胁管理。讲者把它和前面讲过的两个控制点连起来：每个代理应该有唯一标识符，并且访问检查要一路走到最后一跳。威胁管理的问题是：当一个代理突然出现，既没有去 secrets

¹⁰根据 00:10:44–00:12:15 的讲述重绘。

manager，也没有拿到唯一标识符，组织能否实时看见？

这个问题非常实际。若等到审计时才发现“某个代理从未走过秘密管理器、也没有唯一 ID”，事故窗口已经打开很久。威胁管理要求系统在运行时观察异常：代理进程或工作负载出现了，但没有登记；它访问了资源，却没有对应的身份记录；它调用了后端，却没有经过预期秘密管理路径；它在最后一跳触碰敏感资源，却缺少策略判断证据。

审计发现太晚

审计适合证明和复盘，但它经常发生在动作之后。威胁管理要把部分发现前移到运行时：代理刚出现、刚取用秘密、刚访问后端资源、刚绕过身份登记时，就让 CISO、IT 或开发团队看到信号。对高速自动化代理来说，晚几个小时或几天发现，已经足够产生大量错误动作。

这也说明威胁管理不能只看攻击特征。对于 agentic runtime security，很多高风险行为首先表现为身份链路异常，而非传统恶意载荷。没有唯一标识符、绕过 secrets manager、使用共享凭证、最后一跳缺少访问判断，这些都可能成为威胁信号。

6.4 技术合流：编排、治理、态势与威胁管理共同工作

讲者最后把技术栈合在一起：能够做编排、治理和可观测性的引擎，其中可观测性同时覆盖态势管理与威胁管理，跨越整张运行图，才是处理这个问题的方式。这里的“整张图”指前文反复出现的链路：人类身份进入应用，应用触发 AI 代理，代理以非人类身份访问秘密材料、密钥、证书和后端资源，并在最后一跳完成策略判断。

四类能力之间有明确分工。编排把路径跑通，并把身份和会话上下文带到需要的位置；治理把策略压到每一步，确保访问有规则、有判断、有证据；态势管理看清当前身份和敏感材料管理面是否分裂；威胁管理在运行时发现绕过登记、绕过秘密管理器或缺少唯一标识的代理。

可以把它们压缩成一个闭环：

Observe → Govern → Orchestrate → Enforce → Observe

其中，*Observe* 表示通过态势管理和威胁管理获得运行事实；*Govern* 表示把事实转成策略和控制要求；*Orchestrate* 表示把身份、权限、意图和资源访问放进可执行路径；*Enforce* 表示在访问点，特别是最后一跳，做出允许或拒绝；最后再次回到 *Observe*，把新事实反馈给团队和策略。

本讲最后的技术收束

agentic runtime security 的落点超出单个工具名，指向一个运行时闭环：看见代理和非人类身份的真实状态，治理每一步访问，编排人类身份与非人类身份之间的流量，并在最后一跳执行检查。缺少任何一环，企业都很难证明代理行为受控。

6.5 本章小结

本章把可观测性拆成两类能力。态势管理用于盘点和改进当前格局，尤其是 secrets manager、key manager、certificate manager 等管理面的分裂；威胁管理用于实时发现异常代理，例如没有唯一标识符、没有经过 secrets manager、却已经开始访问资源的代理。两者共同服务于上一章提出的跨团队可见性：让 CISO、IT 和开发团队围绕同一张事实图行动。

6.6 总结与延伸

讲者的收束并没有引入新的产品清单或外部合规框架,重点回到本讲主线:要处理 agentic runtime security, 需要能够跨整张运行图工作的编排、治理和可观测性能力;其中可观测性还要同时覆盖态势管理与威胁管理。换句话说,企业要把 AI 代理当成会行动、会拿权限、会触碰后端资源的非人类身份来管理。



图 11: 视频以 IBM 片尾卡结束;前面的讨论把智能体安全收束为身份链路、运行时控制和可观测性三条线。¹¹

整场讲解可以压缩成一条递进链。第一, AI 代理属于非人类身份的一种高风险形态,因为它们运行在代码、容器、函数或虚拟机中,却能代表业务流程行动。第二,传统身份与访问管理覆盖人到应用、甚至人到第一个代理的入口后,后续代理访问后端资源仍然存在断点。第三,断点具体表现为问责缺口、过度授权、委托意图丢失与最后一跳缺少检查,身份冒用是问责和委托链断裂后的典型表现。第四,上线准入要把这些断点变成控制要求:登记代理、剥离常驻权限并动态授予会话权限、绑定意图、在使用点强制执行,并保留完整审计证据。第五,真正落地需要编排、治理和可观测性共同支撑。

从工程角度看,本讲最值得保留的提醒是: AI 代理的风险不只来自模型输出,也来自运行时身份链路。一个代理回答错问题会带来内容风险;一个代理拿错权限、冒用用户、绕过 secrets manager、访问数据库或主机,则会直接进入企业控制面。后者更接近身份与访问管理问题,需要按主体、权限、意图、资源和时间来分析。

读者容易漏掉的一层

很多关于 AI 安全的讨论会集中在提示词、越狱、幻觉和模型评估。本讲关注的是另一层:代理作为工作负载进入企业系统后,怎样被识别、授权、约束、观察和审计。提示词安全解决输入输出边界,运行时身份安全解决“这个代理凭什么能做这件事”的边界。

可以用一个简化问题作为实践检查清单:当某个 AI 代理访问敏感资源时,组织能否回答六个问题?谁触发了它;它自己的唯一标识是什么;它在当前会话拿到了什么权限;它的动作绑定到哪个个人类意图;它在最后一跳访问哪个资源;若事后复盘,能否把人类身份、代理身份、动作、策略判断和响应串成一条证据链。

若这些问题回答不了,说明系统仍然依赖隐含信任:相信代理只会做开发者预期的事,相信应用边界能覆盖代理行为,相信后端访问天然继承前端登录的安全性。讲者的整套框架正是在拆掉这些隐含信任,把代理放回可管理的身份体系里。

¹¹ 视频画面时间区间: 00:12:09–00:12:25。

最终 takeaway 很朴素：agentic AI 的上线门槛不能只看功能是否可用，还要看身份链路是否可解释、权限是否随会话收缩、意图是否可证明、最后一跳是否受控、可观测性是否足够支撑 CISO、IT 和开发三方协作。只有做到这些，AI 代理才有机会在企业环境中承担真实业务动作，同时把非人类身份风险压回可解释、可审计、可处置的范围。

落地前的六问

上线前至少要能连续回答六个问题：谁触发了代理；代理自己的唯一标识是什么；本次会话拿到了哪些权限；动作绑定到哪个人类意图；最后一跳访问了哪个资源；事后能否把人类身份、代理身份、动作、策略判断和响应串成证据链。六问中任何一问断掉，运行时安全就会退回到隐含信任。

上线复盘表

检查项	应该留下的证据
触发者	人类身份、业务上下文和会话入口。
代理身份	代理实例的唯一标识、注册记录和风险量化信息。
会话权限	本次会话授予了哪些权限，以及权限何时收回。
委托意图	人、代理、动作三者之间的绑定关系。
最后一跳	代理访问的数据库、主机、外部系统或其他敏感资源。
审计闭环	策略判断、允许或拒绝结果、响应动作和后续观察记录。

三条落地红线

第一，没有唯一标识的代理不应访问敏感资源，因为后续无法把动作归因到具体实例。第二，拥有常驻宽权限的代理不应进入生产链路，因为权限会脱离会话、意图和风险上下文。第三，缺少使用点检查的后端连接不应被视为已受控，因为代理真正造成影响的位置通常在数据库、主机、外部系统或其他敏感资源入口。三条红线对应本讲反复强调的同一件事：把代理当作会行动的身份来管。

这也是整份笔记的最后落点：agentic AI 的安全工程不能只验收模型输出，还要验收代理在企业系统中留下的身份、权限、意图、资源和审计证据。